```
  def extracted do
    Enum.each(names(), & print_name/1)
  end

  defp print_name(name) do
    capitalized = String.capitalize(name)
    phrase = "Hello, #{capitalized}"
    IO.puts(phrase)
  end

  defp names do
    ["joe", "robert", "mike"]
  end
end
```

These functions are identical in behavior, but the extracted function is easier to understand when read in isolation. This is a personal preference, but the Elixir community generally leans toward extracting multi-line functions.

Now that you've seen how to put functions together, let's go back to modules and look at some of the more advanced convenience-oriented concepts.

Advanced Module Keywords

You won't get too far into an Elixir application without seeing one of the following keywords: alias, require, import, or use. They all serve different purposes and are extensively used in Elixir modules. We'll go over each of them, including when you would use each.

Each of these module keywords can be used as many times as you want in a module. For example, you could alias two modules, require another, and use three more.

This section uses examples that won't actually compile, so you don't need to type them out. They're here to illustrate how the various keywords are used.

Alias

As you create your application, you will have long module names like MyApp.Widgets.Query.WidgetStore. Long module names allow you to cleanly namespace your application, but they are cumbersome to type everywhere. The alias keyword allows you to refer to a module using its shorter name.

```
defmodule ExampleModule do
  alias MyApp.Widgets.Query.WidgetStore
  alias MyApp.Widgets.Workers.WidgetSync
```